SwiftCart
E-Commerce Platform Upgrade

Project Based Learning (PBL) Report

for the course

Software Engineering (20CS31L01)

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

Shaik Haji Nayab Riyaz (24R11A05E2)
Tadoori Satwik (24R11A05E6)

Under the guidance of

Ms. Karunya Sai

Assistant Professor



Department of Computer Science and Engineering
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)
(Affiliated to JNTUH, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal-Malkajgiri Dist.-501 301.

October - 2026

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal-Malkajgiri Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA



CERTIFICATE

This is to certify that the Project-Based Learning (PBL) report entitled **"SwiftCart: E-Commerce Platform Upgrade"** has been carried out by **Shaik Haji Nayab Riyaz (24R11A05E2)**, **Tadoori Satwik (24R11A05E6)**, of III Year I Semester, B.Tech in Computer Science and Engineering, during the academic year 2026-2027, under the guidance of **Karunya Sai, Assistant Professor**, Department of Computer Science and Engineering. This work is submitted in partial fulfillment of the requirements for the Project-Based Learning (PBL) component of the **Software Engineering Lab (20CS31L01)**.

Project Guide	**Head of the Department**
Karunya Sai	**Dr. E. Ravindra**
Assistant Professor	**Professor**

ACKNOWLEDGEMENT

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara(M), Medchal-Malkajgiri Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA

DECLARATION BY THE CANDIDATE

We hereby declare that the Project-Based Learning (PBL) report entitled **"SwiftCart: E-Commerce Platform Upgrade"** submitted to the Department of Computer Science and Engineering, Geethanjali College of Engineering and Technology, is a record of original work carried out by us during the academic year 2026-2027, under the guidance of **Karunya Sai, Assistant Professor,** Department of Computer Science and Engineering. We further declare that this report has not been submitted to any other institution or university for the award of any degree.

Name of the Students:

1. Shaik Haji Nayab Riyaz (24R11A05E2)

2. Tadoori Satwik (24R11A05E6)

Place: Geethanjali College of Engineering and Technology

Date:

TABLE OF CONTENTS

S.No.	Contents	Page No.
1	Abstract	1
2	Introduction	2
3	Problem Statement	3
4	Objectives of Project	4
5	Literature Survey	4
6	System Design / Architecture	5
7	Hardware and Software Requirements	6
8	Implementation	7-14
9	Results and Discussion	15-18
10	Conclusion and Future Scope	19
11	References	20
12	GitHub Repository Link	20

ABSTRACT

SwiftCart is an upgraded e-commerce storefront designed for an online store that is losing customers to slow pages and abandoned shopping carts. Shoppers of the existing store complain that every click reloads the whole page, that the delivery charge appears only at the end, that they are forced to create an account, and that the eight-screen checkout is long and unforgiving. As a result, a large share of carts never become orders. SwiftCart addresses these problems by redesigning how products are browsed, how the cart behaves and how an order is placed.

The project analyses user feedback, ranks the causes of abandonment, and turns them into prioritised requirements and user stories. The resulting design is a single-page storefront that responds without page reloads, shows the full amount payable (items, discount and delivery) in the cart, saves the cart between visits, and replaces the old checkout with a three-step guest checkout on one page. Shoppers can pay by UPI, card or cash on delivery, review each order on a confirmation page, and return later to a “My orders” page that lists every order with its estimated arrival date.

A working prototype was built with HTML, CSS and JavaScript and verified with 65 automated tests (34 unit tests and 31 end-to-end browser tests), all of which pass. The work follows the software engineering life cycle, from requirement analysis and a Software Requirements Specification through UML design, implementation and testing, using an Agile Scrum plan with a user story map and sprint backlog. The design can be extended with a real back end, a payment gateway, order tracking and analytics to measure the drop in cart abandonment after release.

INTRODUCTION

Online shopping depends on a short and predictable path from finding a product to paying for it. When pages are slow, prices change late in the process or the checkout asks for too much, shoppers leave and the sale is lost. SwiftCart is a project to upgrade the storefront of such a store: it keeps the same products and prices, but changes the software so that browsing is fast, costs are clear from the start, and an order can be placed in a few short steps without creating an account.

From a software engineering perspective, the project covers the full development life cycle. User feedback is analysed to identify requirements; improvements are prioritised by impact and effort; requirements are written as a Software Requirements Specification and as user stories; the system is modelled with UML diagrams; a working prototype is implemented; and the prototype is verified with automated unit and end-to-end tests. Agile Scrum is used to plan the work in short sprints so that the most valuable improvements reach shoppers first.

SwiftCart is a fictitious store created for this activity, but the problems it addresses are common to many online stores, especially those whose customers shop on mobile phones. The same approach can be applied to any retail website that suffers from slow pages and cart abandonment. Future enhancements include a production back end with a database, a real payment gateway, order tracking with courier updates, and analytics dashboards that show where shoppers drop out of the checkout.

PROBLEM STATEMENT

An online store receives growing complaints about slow performance, and a large share of the shoppers who add products to the cart leave without paying. The existing storefront reloads the whole page for every action, shows the delivery charge only on the sixth of eight checkout screens, forces every shopper to register and verify an email address, accepts only card payments through a redirect, and loses the cart when the session expires. Each of these issues gives shoppers a reason to abandon their purchase.

The problem is to design an upgraded e-commerce storefront that loads quickly and lets a shopper move from cart to confirmed order in a few short steps, with the full payable amount visible from the start, a choice of payment methods, no forced account creation, and a way to look back at placed orders, so that fewer carts are abandoned. The project aims to streamline the purchase journey, reduce the effort and uncertainty involved in checking out, and give the store the measurements it needs to keep improving.

OBJECTIVES OF THE PROJECT

The main objectives of the SwiftCart project are:

1. To analyse user feedback and identify the main causes of slow performance and cart abandonment.
2. To reduce the checkout from eight screens to a single page with three steps (contact and address, delivery, payment).
3. To show the complete payable amount, including discount and delivery charge, in the cart before checkout starts.
4. To allow shoppers to place an order as a guest, without creating an account.
5. To keep the cart between visits and remind returning shoppers that it was saved.
6. To offer UPI, card and cash on delivery as payment methods, and never store card details.
7. To let shoppers view their placed orders and the estimated arrival date of each.
8. To meet the Core Web Vitals "good" thresholds and cut cart abandonment from an assumed 78% to 65% or lower.

LITERATURE SURVEY

The Baymard Institute reports an average documented online cart abandonment rate of about 70% [1]. The most common reasons shoppers give are extra costs appearing late, being required to create an account, a long or complicated checkout and not seeing the total cost up front [2]. Speed matters as much: 53% of mobile visits are abandoned if a page takes longer than three seconds to load [3], and Google's Core Web Vitals set targets of 2.5 seconds for loading, 200 milliseconds for responsiveness and 0.1 for layout shift [4].

Conventional multi-page checkouts collect sign-in, shipping, billing, delivery and payment on separate screens. Modern single-page guest checkouts ask only for what is needed to deliver the order and validate it as it is entered. SwiftCart applies these findings and, as software engineering texts recommend [5][6], is developed iteratively with Scrum [8].

SYSTEM DESIGN

SwiftCart follows a layered, three-tier client-server architecture. Shoppers interact with the storefront through a web browser, which runs a single-page application: pages change without reloading, and only data is requested from the server. The application tier exposes the catalogue, cart, checkout, payment, delivery and notification functions as services behind an API gateway, and the data tier stores products, orders and carts. A content delivery network (CDN) and a cache keep the storefront fast during sales.

The architecture consists of five layers, as shown in Figure 1. The presentation layer contains the storefront views: catalogue, product page, cart drawer, checkout, confirmation and order history. The API layer routes requests, limits their rate and validates input. The service layer holds one service per business capability, and the domain layer holds the business rules, such as pricing, validation, promotions and the order state machine, with no user-interface code. The data layer uses a relational database for products and orders and an in-memory cache for carts and catalogue data. Security, performance, observability and integration with the payment gateway, courier and email services apply across all layers.

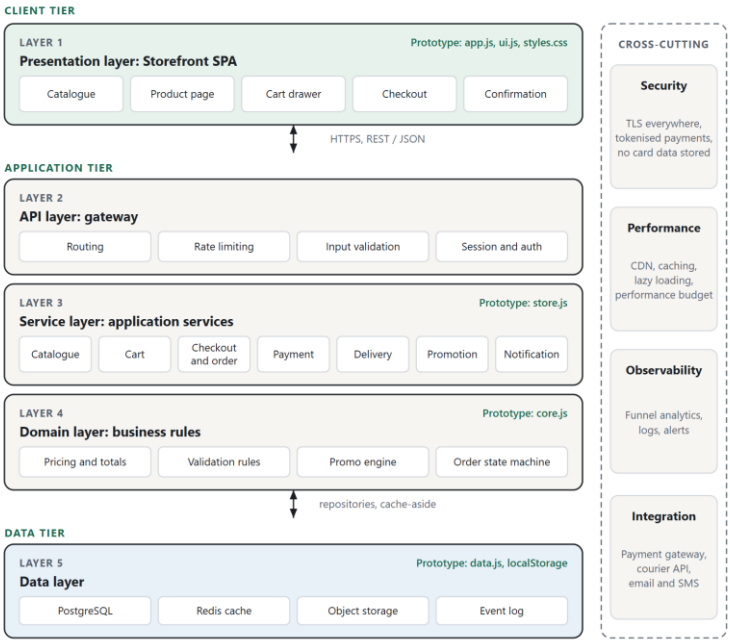


Figure 1: Layered three-tier architecture. Green labels show the prototype file that plays each role.

HARDWARE AND SOFTWARE REQUIREMENTS

Table 1: Hardware requirements

Component	Minimum requirement
Processor	Dual-core processor (1.6 GHz) or equivalent
RAM	4 GB minimum; 8 GB recommended for running the browser tests
Storage	At least 500 MB of free space (prototype under 1 MB; test tools about 35 MB)
Display	1280 × 800 or larger for development; any phone screen from 360 px wide for shoppers
Shopper device	Smartphone, tablet, laptop or desktop with a current web browser
Internet connectivity	Not needed to run the prototype; needed once to install the test tools, and in production

Table 2: Software requirements

Component	Software / Technology
Operating system	Windows 10 or 11, macOS or Linux
Programming languages	HTML5, CSS3, JavaScript (ES2020)
Framework	None: plain JavaScript single-page application with no build step
Data storage (prototype)	Static catalogue file and browser localStorage
Runtime environment	Node.js 18 or later (tests only; 24.18.0 used)
Package manager	npm
Unit testing	Node.js built-in test runner (node:test)
End-to-end testing	puppeteer-core driving Microsoft Edge or Google Chrome
Diagrams and report	SVG diagrams rendered in a headless browser; docx library for this report
Code editor	Visual Studio Code or any suitable editor
Browser	Google Chrome, Microsoft Edge, Firefox or Safari (latest versions)
Version control	Git and GitHub

IMPLEMENTATION

The implementation of SwiftCart follows the Software Development Life Cycle (SDLC), which provides a systematic approach to planning, analysing, designing, developing, testing and maintaining software. Within the life cycle the work is organised with Agile Scrum [8]: requirements are kept in a prioritised product backlog and delivered in two-week sprints, so that the improvements with the greatest effect on abandonment reach shoppers first.

Requirements Gathering and Analysis

The stakeholders are guest shoppers, returning customers, the store owner, the marketing and customer support teams, the operations team that ships orders, and the development team, with the payment gateway and courier as external systems. Because the scenario provides no data, a representative sample of 500 feedback items was assumed from reviews, support tickets, exit surveys and social media, and each item was coded into one theme by affinity mapping.

Slow pages formed the largest single theme (26.4%), but checkout friction was the larger family overall (65.8%). Four themes (slow pages, unexpected costs, forced account creation and a long checkout) account for 72.4% of all feedback, and each was traced to a root cause in the existing store: full page reloads and heavy images, a delivery charge first shown on screen 6 of 8, mandatory registration, and twelve address fields with errors shown only after submitting. Fourteen candidate improvements were scored on an impact/effort matrix and given MoSCoW priorities; guest checkout, costs shown in the cart, a three-step checkout, a saved cart and faster loading were the quick wins chosen for the first release.

Software Requirements Specification (SRS)

The SRS, following the outline of IEEE Std 830 [9], defines the expected behaviour, constraints and quality requirements of SwiftCart and serves as the reference for design and testing. The major functional requirements are:

1. **Catalogue:** List, filter, sort and search products, and show the photo, tax-inclusive price, saving and stock of each.
2. **Cart:** Add products without reloading, change quantities within stock limits, undo removals, and keep the cart between visits.
3. **Transparent pricing:** Show subtotal, discount, delivery charge and total payable in the cart, before checkout starts.
4. **Guest checkout:** Place an order without an account in three steps on one page, with inline validation and PIN-code auto-fill.
5. **Delivery and payment:** Offer standard and express delivery with dates; accept UPI, card or cash on delivery; never store card details; apply promo codes.
6. **Orders:** Confirm each order with its number, items, amount and arrival estimate, and list all placed orders on a My orders page.
7. **Recovery and analytics:** Remind returning shoppers of a saved cart and record checkout funnel events.

The non-functional requirements are page speed within the Core Web Vitals “good” thresholds (LCP $\leq$ 2.5 s, INP $\leq$ 200 ms, CLS $\leq$ 0.1) [4], 99.9% availability of checkout, HTTPS throughout with card data handled only by a PCI-DSS-compliant gateway, accessibility to WCAG 2.1 level AA, support for current browsers on screens from 360 px wide, and business rules kept in one unit-tested module.

Sprint Planning

The requirements were rewritten as 22 user stories with acceptance criteria and story-point estimates, arranged on a user story map [7] and planned over three two-week sprints for a team of four (Table 3).

Table 3: Release plan and sprint backlog

Sprint	Sprint goal	Points
Sprint 1	A guest can add products to a saved cart, see the full amount payable and place an order in three steps.	32
Sprint 2	Instant search, inline validation, PIN auto-fill, saved details, promo codes and three payment methods.	32
Sprint 3	CDN and caching, mobile polish, payment retry, cart reminders, funnel analytics and order history.	31

System Design

SwiftCart uses the layered client-server architecture described in Section 6. The key design change is the checkout workflow: the old eight-screen flow is replaced by a cart drawer that already shows the full total, followed by three steps on one page (contact and address, delivery, payment). Each completed step collapses to an editable summary, payment happens on the same page, and the confirmed order appears on the My orders page.

UML Diagrams

Unified Modeling Language (UML) diagrams represent the structure and behaviour of a software system [10]. The following nine diagrams model SwiftCart.

1. Use Case Diagram: The Shopper browses, manages the cart, checks out as a guest and tracks orders; *Check out as guest* includes contact, delivery and payment and is extended by a promo code. The Store Admin and external systems are also actors.

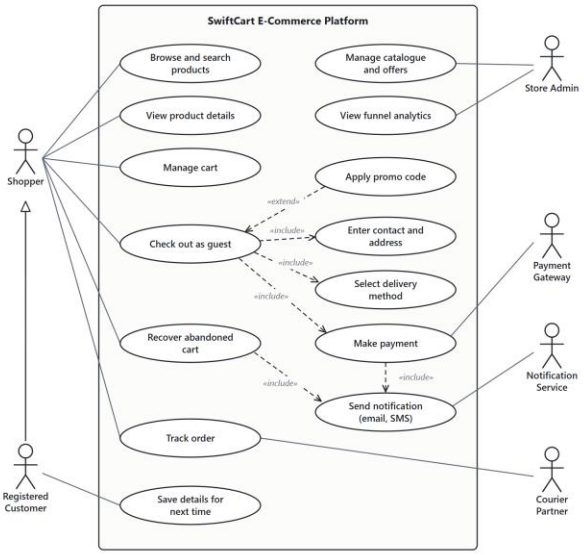


Figure 2: Use case diagram.

2. Class Diagram: A Cart is composed of CartItems that refer to Products. A CheckoutSession creates an Order, whose OrderItems copy product names and prices. Payment is abstract, with one subclass per payment method.

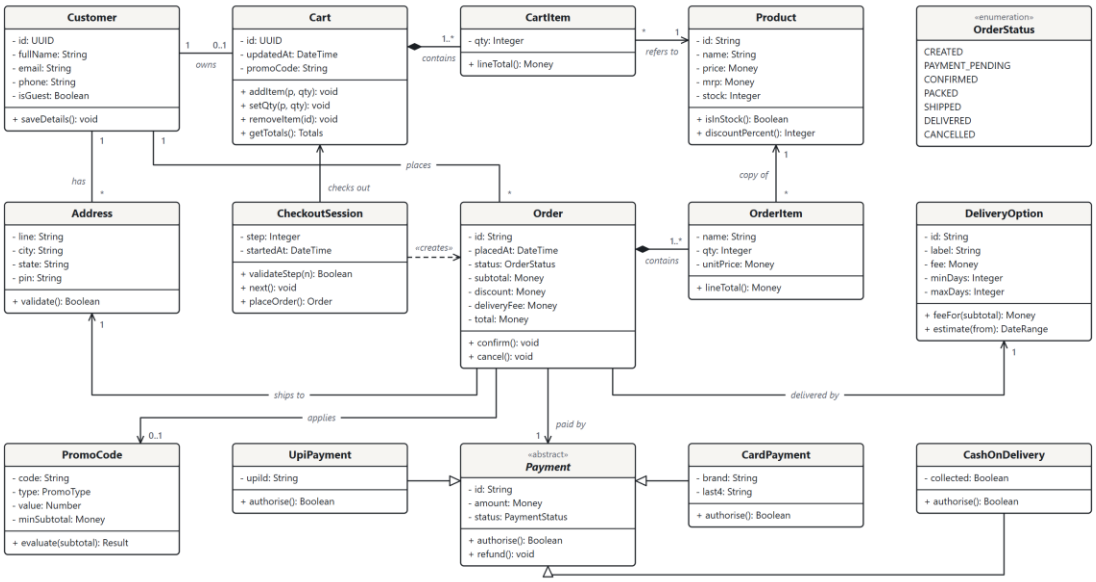


Figure 3: Class diagram.

3. Object Diagram: An example runtime state: one guest order for two items with the WELCOME10 code and free delivery (₹7,398 − ₹500 = ₹6,898), paid by UPI.

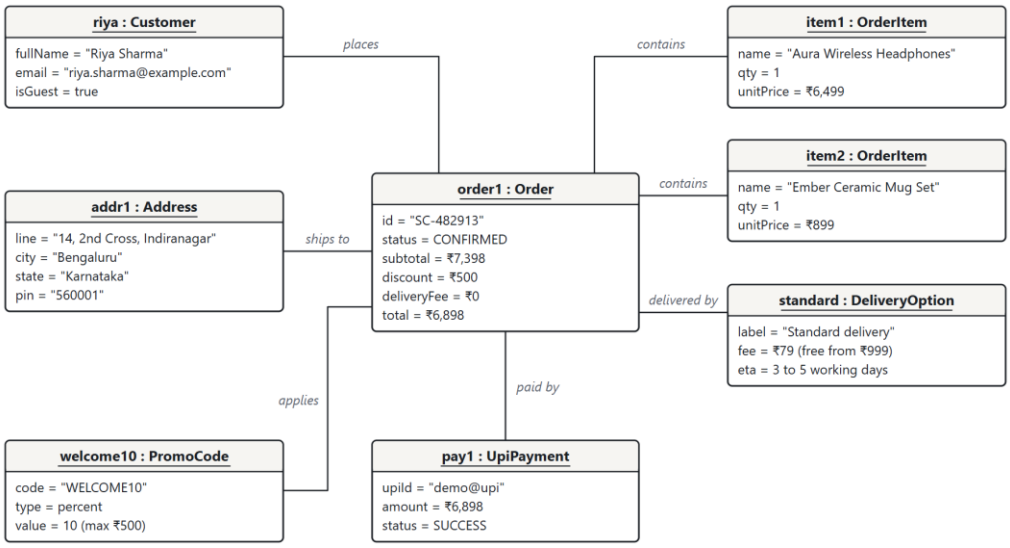


Figure 4: Object diagram of one placed order.

4. Activity Diagram: The enhanced checkout across the shopper, storefront and back-end services. Invalid input and declined payments return to the same step with everything kept.

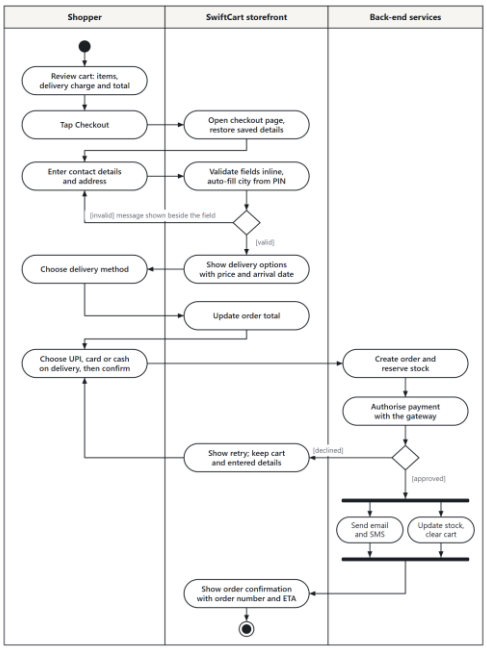


Figure 5: Activity diagram of the checkout.

5. Sequence Diagram: The messages of one guest checkout paid by UPI, with the success and failure outcomes of the payment.

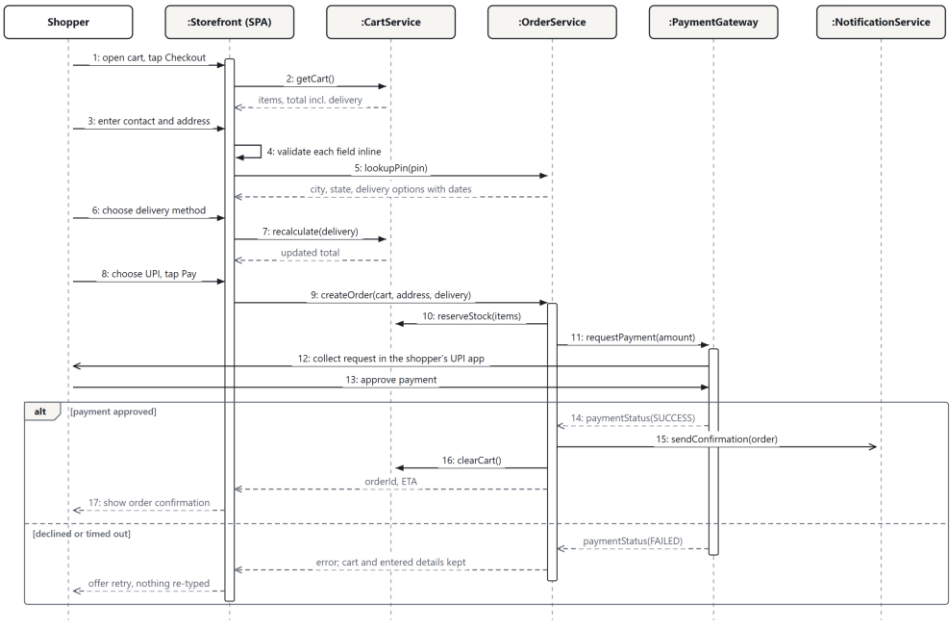


Figure 6: Sequence diagram of a guest checkout.

6. Collaboration Diagram: How the prototype's objects cooperate to place an order and to show the order history.

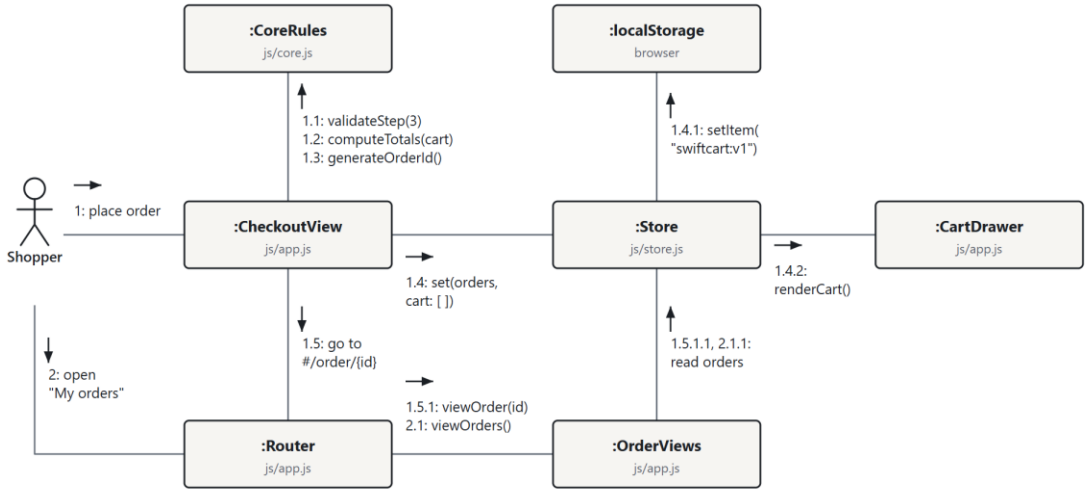


Figure 7: Collaboration diagram of placing and viewing orders.

7. State Chart Diagram: The life cycle of an Order from Created to Delivered; cash on delivery skips payment, and failed payments or timeouts cancel the order.

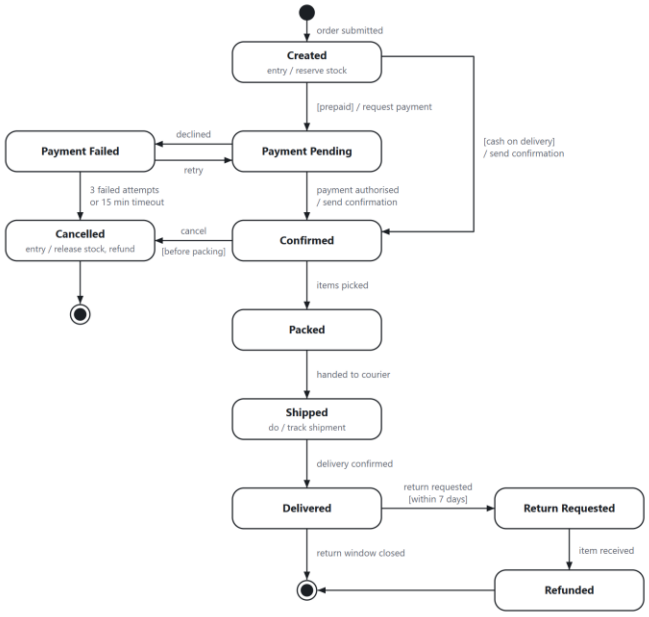


Figure 8: State chart diagram of an Order.

8. Component Diagram: The storefront depends only on the API gateway, which routes to the application services; only three services talk to external systems.

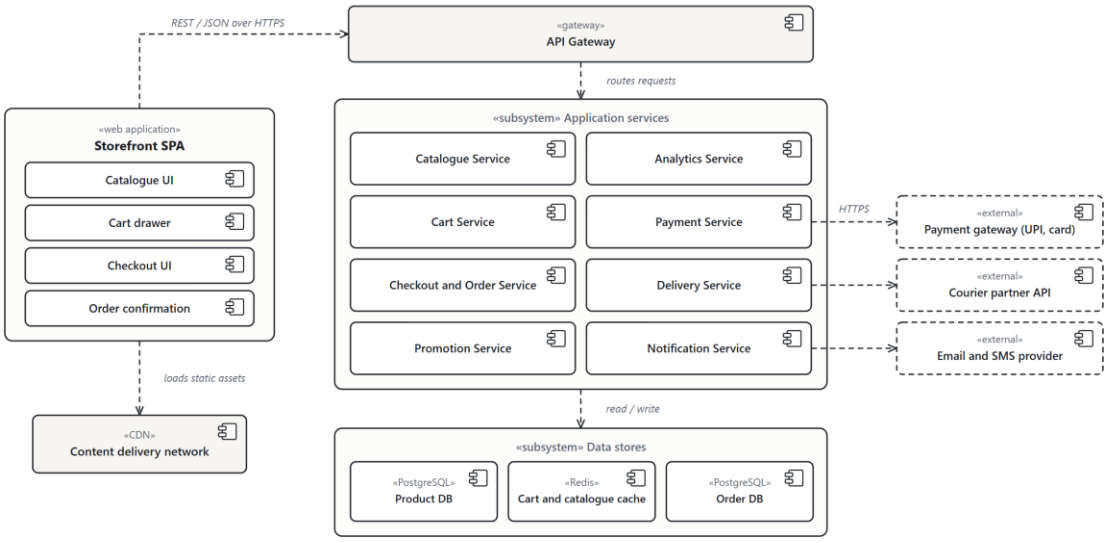


Figure 9: Component diagram.

9. Deployment Diagram: Static files are served from a CDN; API calls pass through a load balancer to scalable application servers backed by Redis and PostgreSQL.

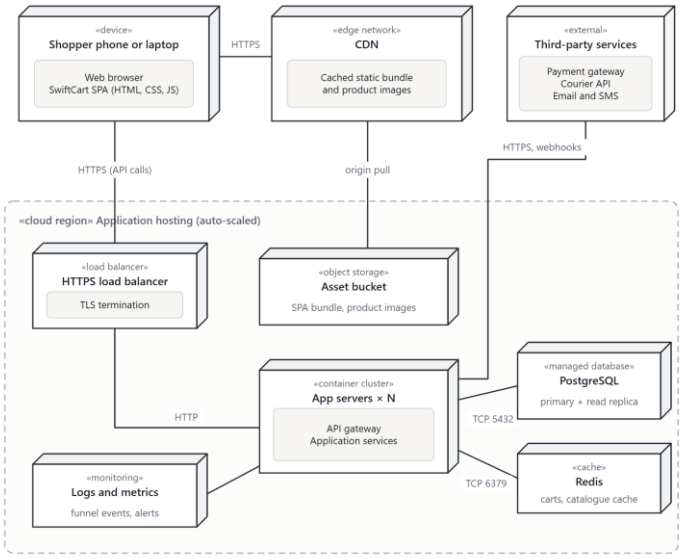


Figure 10: Deployment diagram.

Development and Testing

The prototype is a single-page application written in HTML5, CSS3 and plain JavaScript with no framework or build step, so it runs by opening `prototype/index.html` in any current browser. It is organised in layers: `app.js` and `ui.js` hold the views (catalogue, product page, cart drawer, checkout, confirmation and My orders), `store.js` keeps the cart, saved details and orders in local storage, `core.js` holds the business rules as pure functions, and `data.js` holds the catalogue. Product photos [13] are stored locally and lazy-loaded; payments, the PIN-code service and emails are simulated.

Testing was done at two levels [6]. 34 unit tests, written with the Node.js test runner [11], check the cart, promotion, totals, validation and delivery rules, including boundary values such as an order of exactly ₹999. 31 end-to-end tests drive the prototype in Microsoft Edge with puppeteer-core [12] at desktop and phone sizes, covering the catalogue, cart, checkout, all three payment methods, saved details, script injection, the mobile layout, analytics events and the My orders page.

RESULTS AND DISCUSSION

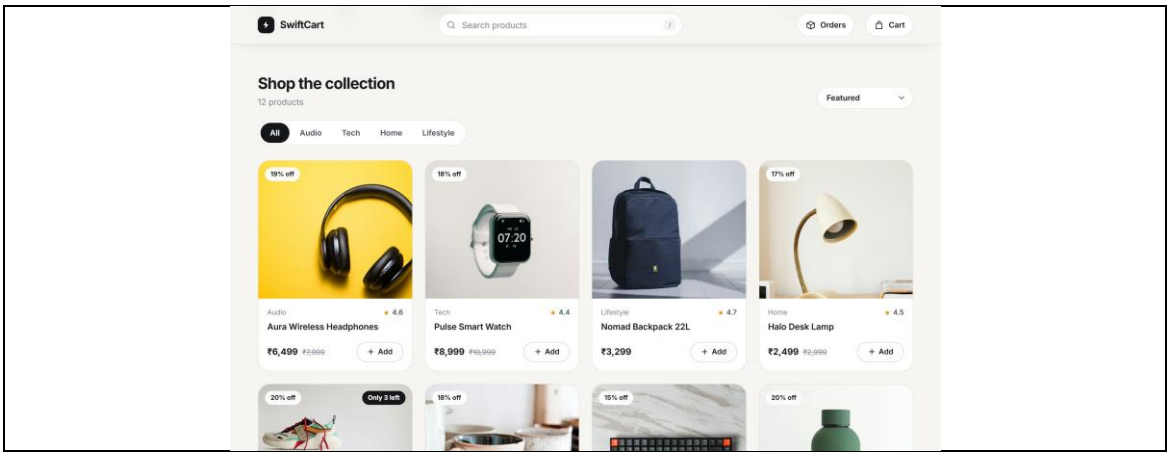


Figure 11: Catalogue with product photos, filters, tax-inclusive prices and one-tap Add buttons.

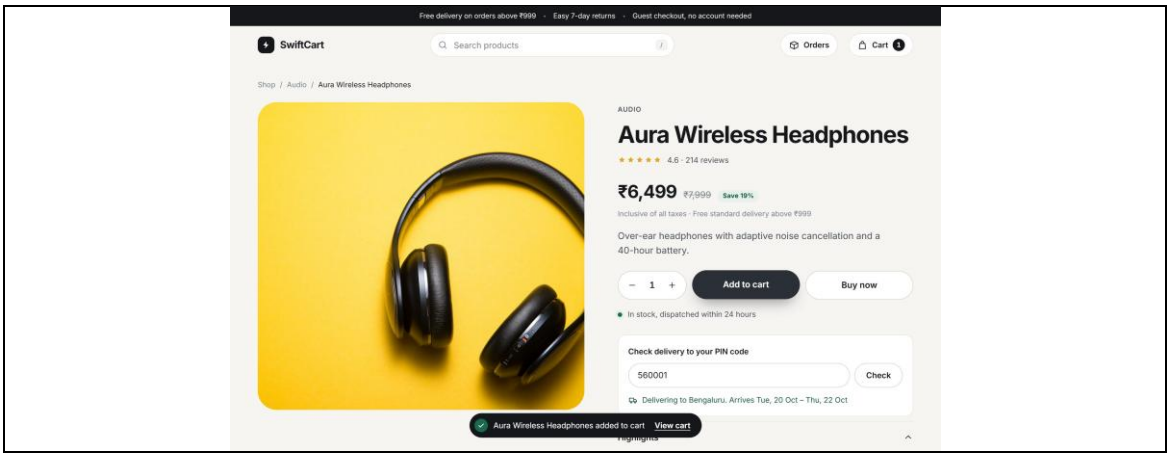


Figure 12: Product page with saving, stock status and delivery check by PIN code.

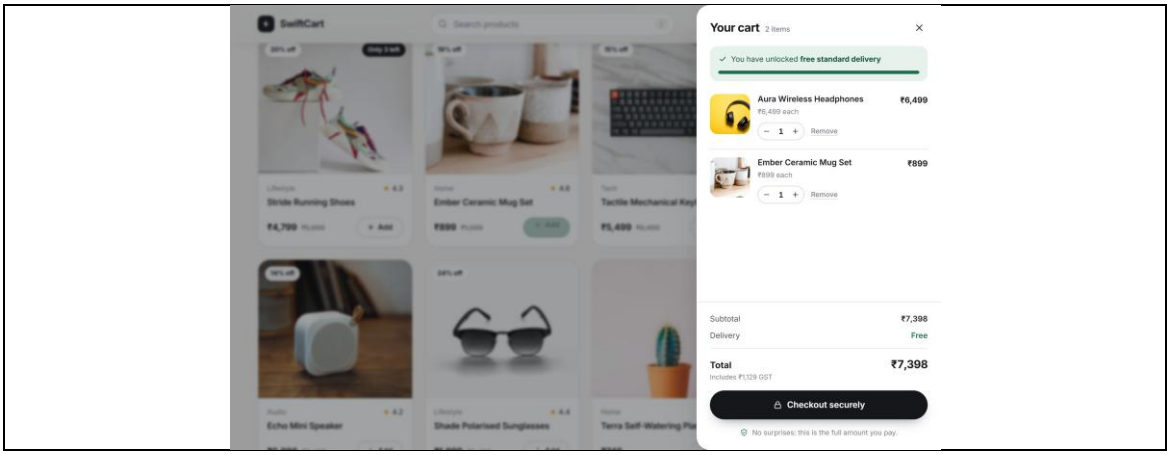


Figure 13: Cart drawer showing free-delivery progress and the full amount payable.

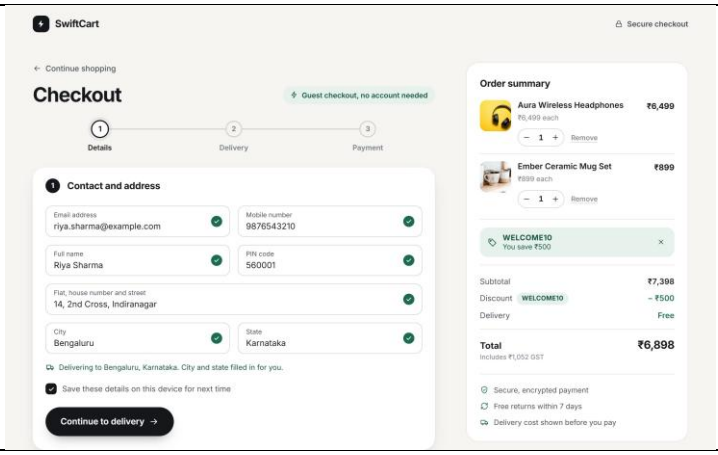


Figure 14: Three-step checkout: city and state filled in from the PIN code, promo code applied.

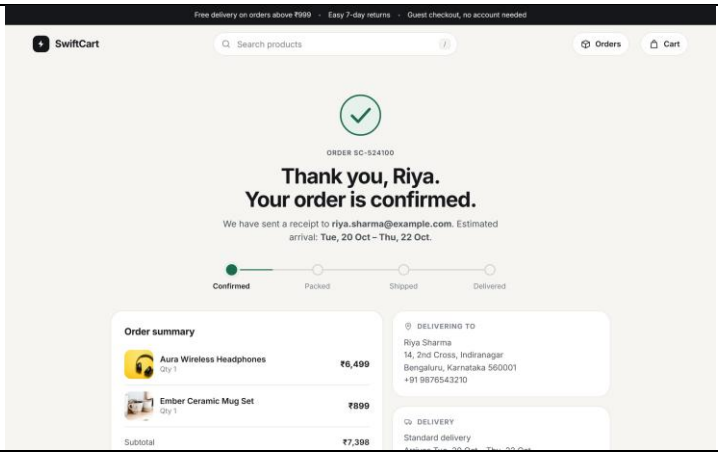


Figure 15: Order confirmation with order number, arrival estimate and status timeline.

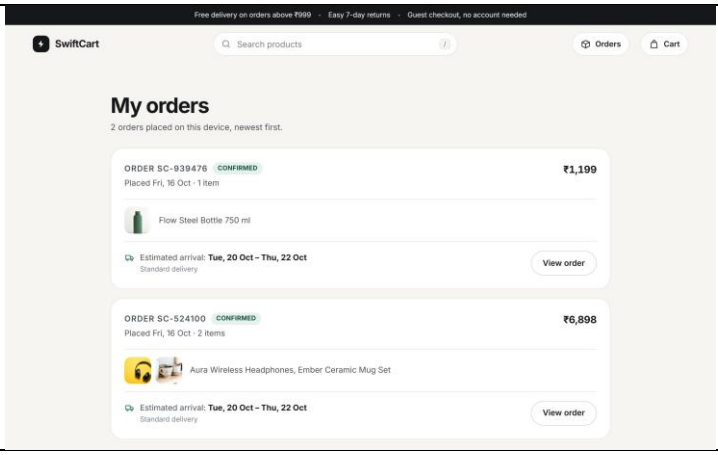


Figure 16: My orders: every placed order with its total, items and estimated arrival.

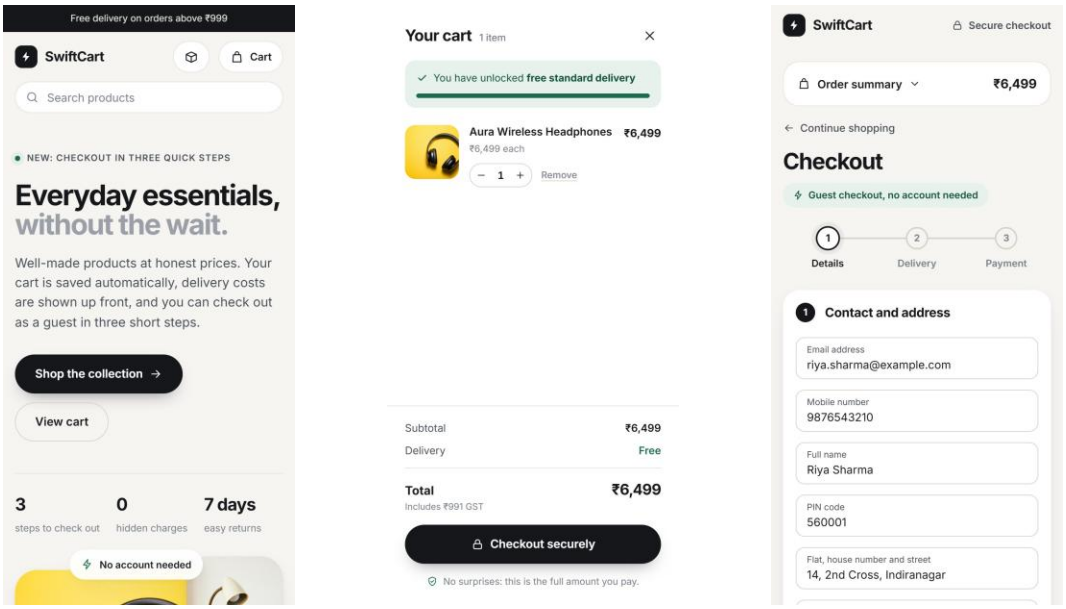


Figure 17: The prototype at phone width (390 px): home page, cart and checkout.

Both test suites were run on Windows 11 with Node.js 24.18.0 and Microsoft Edge. All 65 tests passed (Table 4). Testing also found two real defects earlier in development, a fading backdrop that swallowed the next click and a message that could cover an Add button; both were fixed and retested.

Table 4: Summary of the final test run

Suite	Tests	Passed	Failed	Duration
Unit tests	34	34	0	0.2 s
End-to-end tests	31	31	0	61.1 s
Total	**65**	**65**	**0**	

Table 5 compares the checkout of the existing store (assumed baseline) with the prototype (measured).

Table 5: Checkout before and after the redesign

Aspect	Before	After
Screens from cart to order	8	1 page, 3 steps
Account required	Yes, with email verification	No
Required contact and address fields	12, plus billing address	7, two filled in from the PIN code
Delivery charge first shown	Screen 6	In the cart
Payment methods	Card (redirect)	UPI, card, cash on delivery
Cart after a return visit	Lost	Kept, with a reminder
Viewing past orders	Only after signing in	My orders page, no account needed

The prototype meets the objectives that concern the design of the checkout and the order history, and partly meets the cart-reminder objective, since email and SMS reminders need a back end. Page speed in the field and the abandonment rate can only be confirmed with real shoppers; the front end is 145 KB of code plus 642 KB of lazy-loaded photos, and an A/B test after release would compare the completion rate of the old and new checkouts.

CONCLUSION AND FUTURE SCOPE

SwiftCart demonstrates how software engineering practice can turn customer complaints into a better product. Analysing user feedback showed that nearly three-quarters of the complaints came from four causes: pages that reload for every action, costs that appear late, a forced account and a long, unforgiving checkout. The upgraded storefront addresses each cause with a single-page design, a cart that shows the full amount payable and is saved automatically, a three-step guest checkout with inline validation, a choice of UPI, card or cash on delivery, and a My orders page with arrival estimates.

The project applied the full development life cycle, from requirement analysis and an SRS through sprint planning, nine UML diagrams and a layered architecture to a working prototype verified by 65 automated tests. The current version has limitations: the feedback data and baseline figures are assumed, orders are kept only in the browser, payments and emails are simulated, and testing covered one browser engine. It is therefore suited to demonstration and academic purposes.

Future enhancements include a back end with a database for products, carts and orders, a real payment gateway with payment retry, user accounts that keep order history across devices, courier tracking, email and SMS cart reminders, a CDN for speed during sales, and analytics dashboards that show where shoppers drop out. With these in place, an A/B test after release would measure the actual reduction in cart abandonment.

REFERENCES

1. Baymard Institute. *Cart Abandonment Rate Statistics*. Available at: https://baymard.com/lists/cart-abandonment-rate
2. OptinMonster. *Cart Abandonment Statistics*. Available at: https://optinmonster.com/cart-abandonment-statistics/
3. Google. *The Need for Mobile Speed*, as reported by Marketing Dive. Available at: https://www.marketingdive.com/news/google-53-of-mobile-users-abandon-sites-that-take-over-3-seconds-to-load/426070/
4. Google. *Web Vitals*. Available at: https://web.dev/articles/vitals
5. Sommerville, I. *Software Engineering*, 10th ed. Pearson, 2016.
6. Pressman, R. S., and Maxim, B. R. *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2015.
7. Patton, J. *User Story Mapping*. O'Reilly Media, 2014.
8. Schwaber, K., and Sutherland, J. *The Scrum Guide*, 2020. Available at: https://scrumguides.org
9. IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*, 1998.
10. Object Management Group (OMG). *Unified Modeling Language (UML) Specification*. Available at: https://www.omg.org/spec/UML/
11. Node.js. *Test Runner Documentation*. Available at: https://nodejs.org/api/test.html
12. Puppeteer. *Puppeteer Documentation*. Available at: https://pptr.dev
13. Unsplash. *Unsplash License*. Available at: https://unsplash.com/license

GITHUB REPOSITORY

GitHub Repository Link: [GitHub repository link]